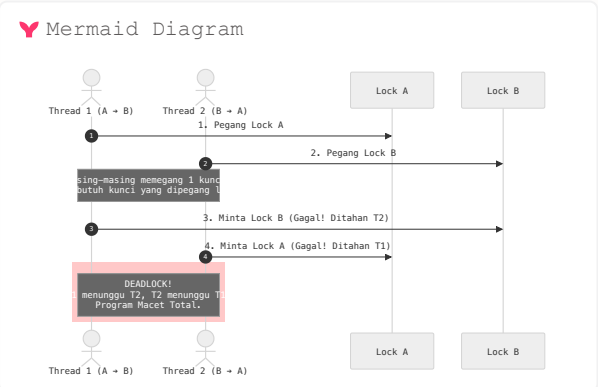
#7 Deadlock, Livelock



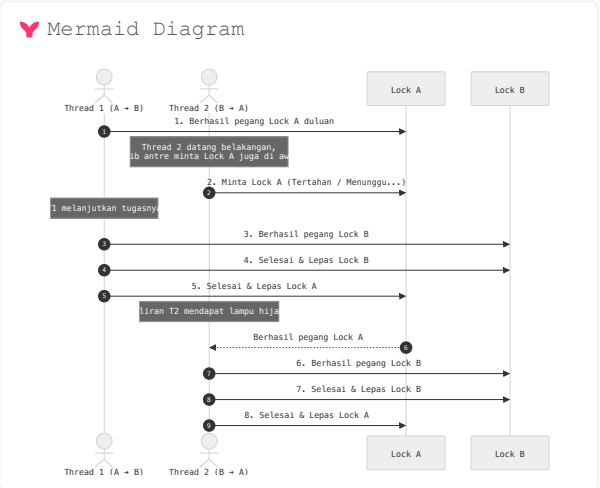
Deadlock

A deadlock occurs when a group of **processes or threads is stuck because each one is waiting for an event that only another member of the same group can cause**, usually the release of a resource it is holding. Since everyone is waiting, nobody ever releases anything, and the whole group stops making progress permanently while consuming no CPU.

Deadlock



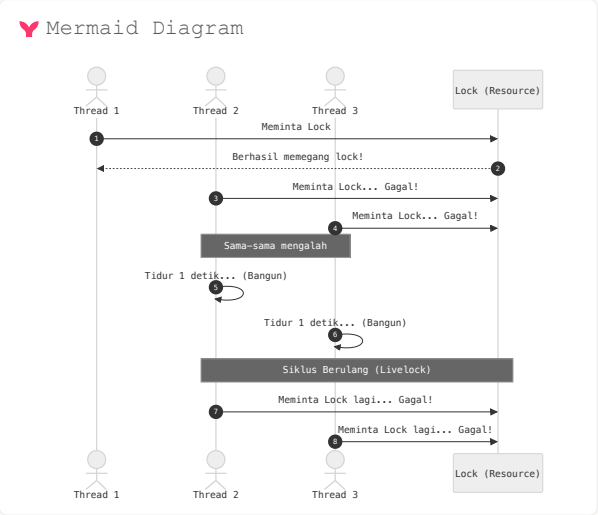
Lock ordering



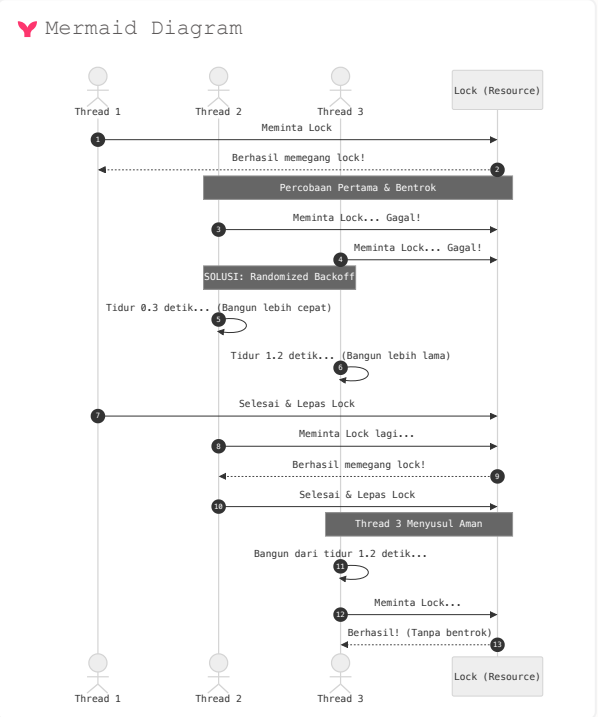
Livelock

A livelock occurs when two or more processes keep changing their state in response to one another, yet none of them makes real progress. Unlike a deadlock, the processes are not blocked; they are busy running and using CPU time, but their effort is wasted, much like two people in a corridor who both step aside in the same direction again and again.

Livelock



Randomized backoff



Labs

`deadlock.py`

Shell

```
import threading
import time

saldo_rekening = {
    "Rekening_A": 1000000,
    "Rekening_B": 500000,
    "Rekening_C": 750000
}

lock_a = threading.Lock()
lock_b = threading.Lock()
lock_c = threading.Lock()

def transfer_dana_1():
    print("Thread 1 (Transfer A -> B): Meminta kunci Rekening A...")
    lock_a.acquire()
    print("Thread 1 (Transfer A -> B): Berhasil mengunci Rekening A.")

    time.sleep(1) # Simulasi jeda / proses verifikasi data

    print("Thread 1 (Transfer A -> B): Meminta kunci Rekening B...")
    lock_b.acquire() # <--- DEADLOCK TERJADI DI SINI JIKA KUNCI SEDANG
DIPEGANG THREAD 2
    print("Thread 1 (Transfer A -> B): Berhasil mengunci Rekening B.")

    # Proses Transfer (Critical Section)
    print("Thread 1: Melakukan transfer saldo A ke B...")

    # Melepaskan kunci
    lock_b.release()
    lock_a.release()
    print("Thread 1: Transfer selesai, kunci dilepaskan.")

def transfer_dana_2():
    """
    Worker 2: Transfer dari Rekening B ke Rekening C
    Urutan penguncian terbalik: Kunci Rekening B dulu, baru Rekening C
    (Catatan: Untuk memicu deadlock murni dengan Thread 1, kita buat
Worker 2
    memegang B dan meminta A, atau mensimulasikan persilangan akses ke
Rekening B)
    """
    print("Thread 2 (Transfer B -> A): Meminta kunci Rekening B...")
    lock_b.acquire()
    print("Thread 2 (Transfer B -> A): Berhasil mengunci Rekening B.")
```

```
        time.sleep(1) # Simulasi jeda

        print("Thread 2 (Transfer B -> A): Meminta kunci Rekening A...")
        lock_a.acquire() # <--- DEADLOCK TERJADI DI SINI KARENA SALING TUNGGU
    DENGAN THREAD 1
        print("Thread 2 (Transfer B -> A): Berhasil mengunci Rekening A.")

        # Proses Transfer (Critical Section)
        print("Thread 2: Melakukan transfer saldo B ke A...")

        # Melepaskan kunci
        lock_a.release()
        lock_b.release()
        print("Thread 2: Transfer selesai, kunci dilepaskan.")

    # Menjalankan dua thread transfer secara bersamaan
    t1 = threading.Thread(target=transfer_dana_1)
    t2 = threading.Thread(target=transfer_dana_2)

    t1.start()
    t2.start()

    t1.join()
    t2.join()

    print("Semua proses transfer selesai (Baris ini tidak akan pernah tercetak
    karena Deadlock!).")
```

`solusi_deadlock.py`

Python

```
import threading
import time

saldo_rekening = {
    "Rekening_A": 1000000,
    "Rekening_B": 500000,
    "Rekening_C": 750000
}

lock_a = threading.Lock()
lock_b = threading.Lock()
lock_c = threading.Lock()

def transfer_dana_1():
    print("Thread 1 (Transfer A -> B): Meminta kunci Rekening A...")
    lock_a.acquire()
    print("Thread 1 (Transfer A -> B): Berhasil mengunci Rekening A.")

    time.sleep(1)

    print("Thread 1 (Transfer A -> B): Meminta kunci Rekening B...")
    lock_b.acquire()
    print("Thread 1 (Transfer A -> B): Berhasil mengunci Rekening B.")

    print("Thread 1: Melakukan transfer saldo A ke B...")
    saldo_rekening["Rekening_A"] -= 100000
    saldo_rekening["Rekening_B"] += 100000

    lock_b.release()
    lock_a.release()
    print("Thread 1: Transfer selesai, kunci dilepaskan.\n")

def transfer_dana_2():
    print("Thread 2 (Transfer B -> A): Meminta kunci Rekening A
(di awal)...")
    lock_a.acquire()
    print("Thread 2 (Transfer B -> A): Berhasil mengunci Rekening A.")

    time.sleep(1)
    print("Thread 2 (Transfer B -> A): Meminta kunci Rekening B...")
    lock_b.acquire()
    print("Thread 2 (Transfer B -> A): Berhasil mengunci Rekening B.")

    print("Thread 2: Melakukan transfer saldo B ke A...")
    saldo_rekening["Rekening_B"] -= 50000
    saldo_rekening["Rekening_A"] += 50000
```

```
        lock_b.release()
        lock_a.release()
        print("Thread 2: Transfer selesai, kunci dilepaskan.\n")

    t1 = threading.Thread(target=transfer_dana_1)
    t2 = threading.Thread(target=transfer_dana_2)

    t1.start()
    t2.start()

    t1.join()
    t2.join()

    print("Semua proses transfer selesai dengan aman tanpa Deadlock!")
    print("Saldo Akhir:", saldo_rekening)
```

`livelock.py`

Python

```
import threading
import time

resource_lock = threading.Lock()

def thread_1_worker():
    print("Thread 1: Meminta Lock...")
    resource_lock.acquire()
    print("Thread 1: Berhasil memegang lock! (Sedang bekerja selama
3 detik...)")
    try:
        time.sleep(3.0)
    finally:
        resource_lock.release()
        print("Thread 1: Lock dilepaskan.")

def thread_2_worker():
    time.sleep(0.1)

    print("Thread 2: Meminta Lock...")
    if not resource_lock.acquire(blocking=False):
        print("Thread 2: Gagal! (Lock sedang dipegang Thread lain)")

    print("Thread 2: Tidur 1 detik... (Mengalah)")
    time.sleep(1.0)

    print("Thread 2: Meminta Lock lagi...")
    if not resource_lock.acquire(blocking=False):
        print("Thread 2: Gagal lagi! (Resource sibuk)")

def thread_3_worker():
    time.sleep(0.1)

    print("Thread 3: Meminta Lock...")
    if not resource_lock.acquire(blocking=False):
        print("Thread 3: Gagal! (Lock sedang dipegang Thread lain)")

    print("Thread 3: Tidur 1 detik... (Mengalah)")
    time.sleep(1.0)

    print("Thread 3: Meminta Lock lagi...")
    if not resource_lock.acquire(blocking=False):
        print("Thread 3: Gagal lagi! (Resource sibuk)")

if __name__ == "__main__":
    # Menjalankan thread sesuai diagram
```

```
    t1 = threading.Thread(target=thread_1_worker, name="T1")
    t2 = threading.Thread(target=thread_2_worker, name="T2")
    t3 = threading.Thread(target=thread_3_worker, name="T3")

    t1.start()
    t2.start()
    t3.start()

    t1.join()
    t2.join()
    t3.join()

    print("Simulasi selesai.")
```

`solusi_livelock.py`

Python

```
import threading
import time
import random

resource_lock = threading.Lock()

def worker(nama_thread, backoff_detik=None):
    print(f"{nama_thread}: Meminta Lock...")

    if not resource_lock.acquire(blocking=False):
        print(f"{nama_thread}: Meminta Lock... Gagal! (Resource sibuk)")

    waktu_tunggu = backoff_detik if backoff_detik else
random.uniform(0.2, 1.5)
    print(f"{nama_thread}: Tidur selama {waktu_tunggu:.1f} detik
(Randomized Backoff)...")
    time.sleep(waktu_tunggu)

    print(f"{nama_thread}: Bangun dan Meminta Lock lagi...")
    while not resource_lock.acquire(blocking=False):
        time.sleep(0.1)

    print(f"{nama_thread}: Berhasil memegang lock!")
    try:
        time.sleep(0.3)
    finally:
        resource_lock.release()
        print(f"{nama_thread}: Selesai & Lepas Lock.")

if __name__ == "__main__":
    def thread_1_owner():
        print("Thread 1: Meminta Lock...")
        resource_lock.acquire()
        print("Thread 1: Berhasil memegang lock!")
        try:
            time.sleep(1.0)
        finally:
            resource_lock.release()
            print("Thread 1: Selesai & Lepas Lock.")

    t1 = threading.Thread(target=thread_1_owner, name="T1")
    t2 = threading.Thread(target=worker, args=("Thread 2", 0.3)) # T2
bangun cepat
    t3 = threading.Thread(target=worker, args=("Thread 3", 1.2)) # T3
bangun lama
```

```
    t1.start()
    t2.start()
    t3.start()

    t1.join()
    t2.join()
    t3.join()

    print("\nSimulasi selesai dengan sukses!")
```